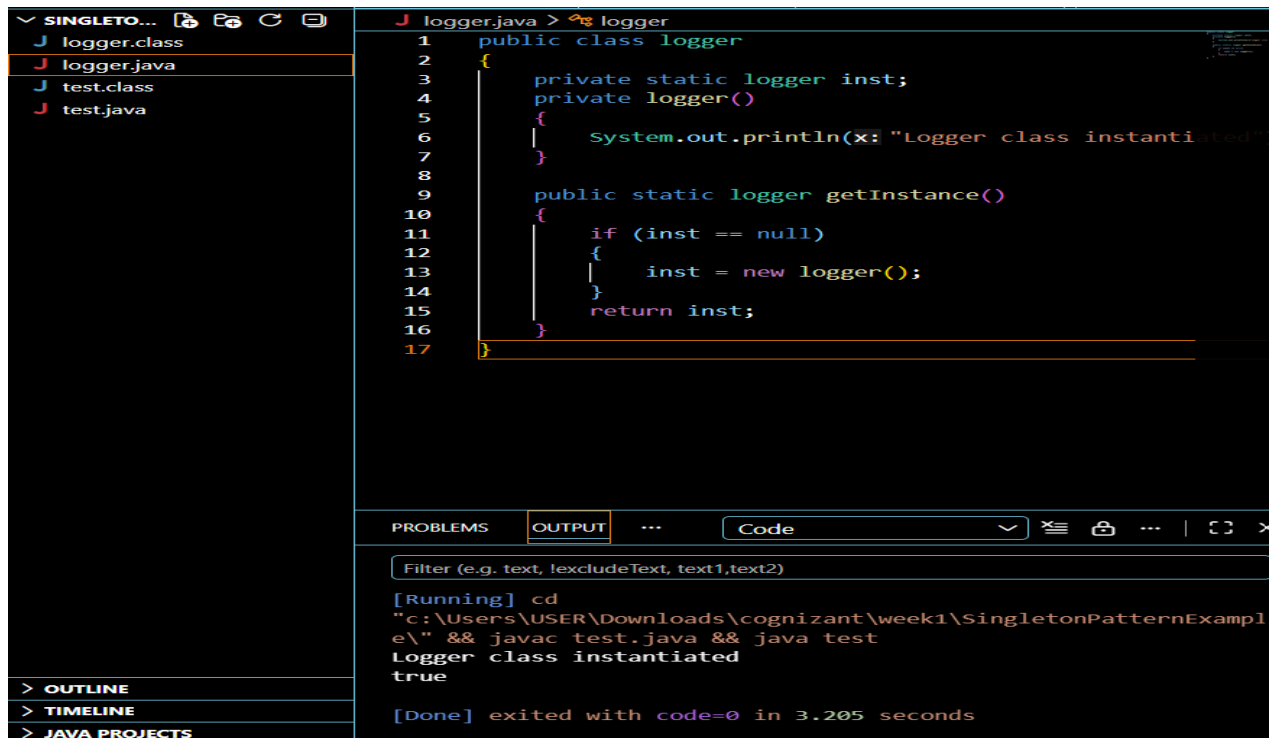


Exercise 1: Implementing the Singleton Pattern

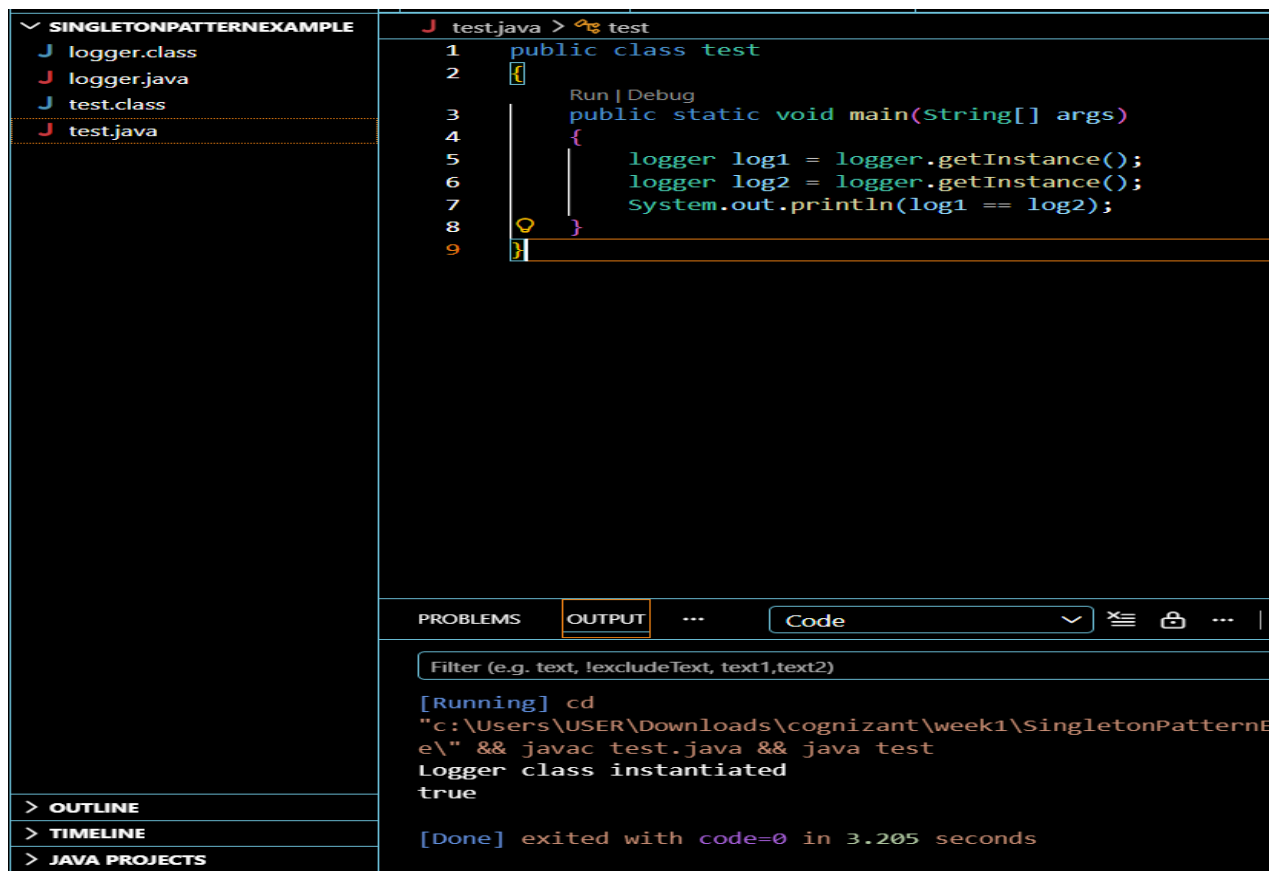


The screenshot shows an IDE with a project named "SINGLETONPATTERNEXAMPLE". The file explorer on the left lists "logger.class", "logger.java", "test.class", and "test.java". The main editor displays the code for "logger.java":

```
1 public class logger
2 {
3     private static logger inst;
4     private logger()
5     {
6         System.out.println(x: "Logger class instantiated")
7     }
8
9     public static logger getInstance()
10    {
11        if (inst == null)
12        {
13            inst = new logger();
14        }
15        return inst;
16    }
17 }
```

Below the code editor, the "OUTPUT" tab shows the following text:

```
[Running] cd
"c:\Users\USER\Downloads\cognizant\week1\SingletonPatternExample\" && javac test.java && java test
Logger class instantiated
true
[Done] exited with code=0 in 3.205 seconds
```



The screenshot shows the same IDE with the project "SINGLETONPATTERNEXAMPLE". The file explorer on the left lists "logger.class", "logger.java", "test.class", and "test.java". The main editor displays the code for "test.java":

```
1 public class test
2 {
3     Run | Debug
4     public static void main(String[] args)
5     {
6         logger log1 = logger.getInstance();
7         logger log2 = logger.getInstance();
8         System.out.println(log1 == log2);
9     }
10 }
```

Below the code editor, the "OUTPUT" tab shows the following text:

```
[Running] cd
"c:\Users\USER\Downloads\cognizant\week1\SingletonPatternExample\" && javac test.java && java test
Logger class instantiated
true
[Done] exited with code=0 in 3.205 seconds
```

Exercise 2: Implementing the Factory Method Pattern

```
1 public class WordDocument implements Document
2 {
3     @Override
4     public void open()
5     {
6         System.out.println("Opening word document");
7     }
8 }
```

```
1 public class ExcelDocument implements Document
2 {
3     @Override
4     public void open()
5     {
6         System.out.println("Opening excel document");
7     }
8 }
```

```
1 public class PdfDocument implements Document
2 {
3     @Override
4     public void open()
5     {
6         System.out.println("Opening PDF document");
7     }
8 }
```

```
1 public class WordFactory extends DocumentFactory
2 {
3     @Override
4     public Document createDocument()
5     {
6         return new WordDocument();
7     }
8 }
```

```
1 public class ExcelFactory extends DocumentFactory
2 {
3     @Override
4     public Document createDocument()
5     {
6         return new ExcelDocument();
7     }
8 }
```

```
1 public class PdfFactory extends DocumentFactory
2 {
3     @Override
4     public Document createDocument()
5     {
6         return new PdfDocument();
7     }
8 }
```

J ExcelDocument.java	J ExcelFactory.java	J Test.java X	▶ ▼
----------------------	---------------------	---------------	-----

```
J Test.java
1  public class Test
2  {
3      public static void main(String[] args)
4      {
5          DocumentFactory pdfFactory = new PdfFactory();
6          Document pdfDocument = pdfFactory.createDocument();
7          pdfDocument.open();
8
9          DocumentFactory wordFactory = new WordFactory();
10         Document wordDocument = wordFactory.createDocument();
11         wordDocument.open();
12
13         DocumentFactory excelFactory = new ExcelFactory();
14         Document excelDocument = excelFactory.createDocument();
15         excelDocument.open();
16     }
17 }
```

```
PS C:\Users\USER\Downloads\cognizant\week1> cd .\FactoryMethod\
PS C:\Users\USER\Downloads\cognizant\week1\FactoryMethod> javac *.java
PS C:\Users\USER\Downloads\cognizant\week1\FactoryMethod> java Test
Opening PDF document
Opening word document
Opening excel document
PS C:\Users\USER\Downloads\cognizant\week1\FactoryMethod> |
```

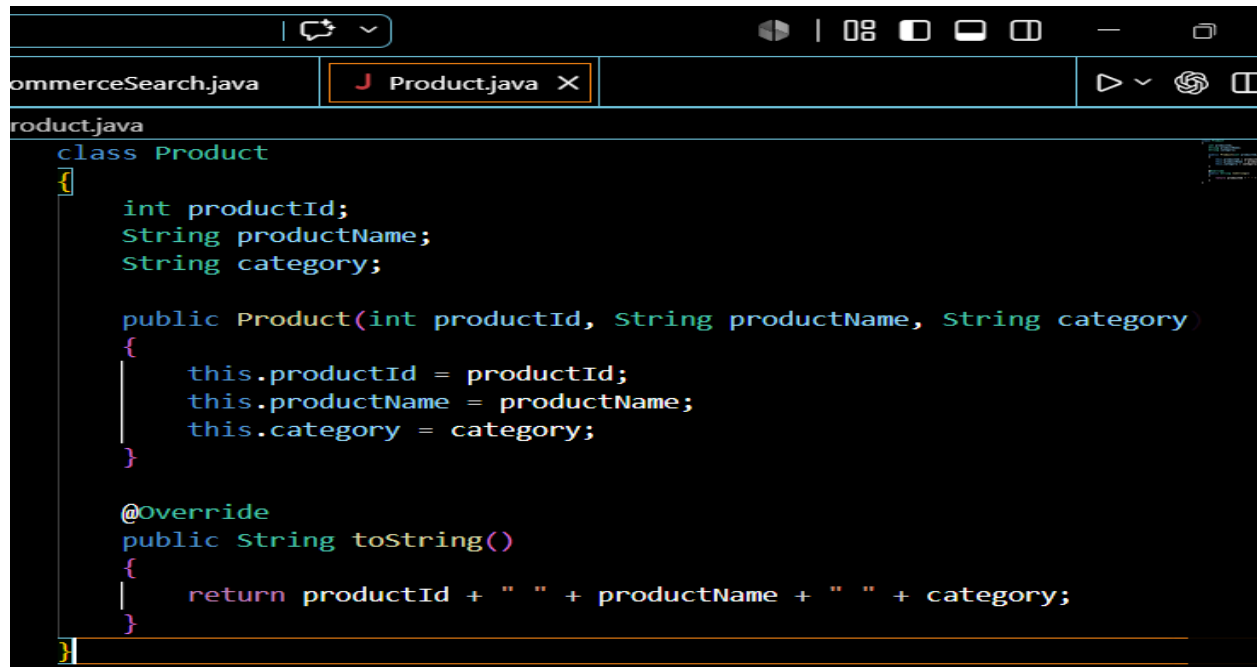
Exercise 2: E-commerce Platform Search Function

J Linear.java	J EcommerceSearch.java
<pre>1 public static Product linearSearch(Product[] products, int id) 2 { 3 for (Product product : products) 4 { 5 if (product.productId == id) 6 { 7 return product; 8 } 9 } 10 return null; 11 } 12</pre>	<pre>1 <u>lic</u> class EcommerceSearch 2 3 public static Product linearSearch(Product[] products, int id) 4 { 5 for (Product product : products) 6 { 7 if (product.productId == id) 8 { 9 return product; 10 } 11 } 12 return null; 13 } 14</pre>

The screenshot shows an IDE interface with the following components:

- Explorer Panel:** Lists files under the **ECOMMERCE** project: Binary.java, EcommerceSearch.class, EcommerceSearch.java, Linear.java, Product.class, and Product.java.
- File Tabs:** Includes 'Welcome', 'Linear.java', and 'Binary.java' (which is the active file).
- Code Editor:** Displays the code for **Binary.java**, which implements a binary search algorithm.

```
1 public static Product binarySearch(Product[] products, int id)
2 {
3     int low=0;
4     int high=products.length-1;
5     while (low<=high)
6     {
7         int mid=(low+high)/2;
8         if (products[mid].productId==id)
9         {
10            return products[mid];
11        }
12        else if (products[mid].productId<id)
13        {
14            low=mid+1;
15        }
16        else
17        {
18            high=mid-1;
19        }
20    }
21    return null;
22 }
```



```
Product.java
class Product
{
    int productId;
    String productName;
    String category;

    public Product(int productId, String productName, String category)
    {
        this.productId = productId;
        this.productName = productName;
        this.category = category;
    }

    @Override
    public String toString()
    {
        return productId + " " + productName + " " + category;
    }
}
```

```
[Running] cd "c:\Users\USER\Downloads\cognizant\week1\ecommerce\" && javac EcommerceSearch.java && java EcommerceSearch
Linear Search Result:
103 Watch Accessories
Binary Search Result:
103 Watch Accessories

[Done] exited with code=0 in 0.775 seconds
```

Exercise 7: Financial Forecasting

```
1 public class FinancialForecast
2 {
3     public static double forecast( double currentValue, double growthRate, int years)
4     {
5         if( years==0)
6         {
7             return currentValue;
8         }
9     }
10    return forecast( currentValue * ( 1+growthRate), growthRate, years-1);
11 }
12
13
14 public static void main(String args[])
15 {
16     double present = 1000.0;
17     double growthRate = 0.05;
18     int years = 5;
19     double ft= forecast(present, growthRate, years);
20     System.out.println("Forecasted value: " + ft);
21 }
22 }
```

[Running] cd "c:\Users\USER\Downloads\cognizant\week1\financialForecasting\" && javac FinancialForecast.class
Forecasted value: 1276.2815625000003
[Done] exited with code=0 in 1.747 seconds

Exercise 1: Control Structures

Scenario 1:

DECLARE

BEGIN

FOR R IN (

SELECT L.LOANID, L.RATE

FROM LOANS L JOIN CUSTOMERS C

ON L.CUSTOMERID = C.CUSTOMERID

WHERE C.AGE>60

```

        )
    LOOP
        UPDATE LOANS
        SET RATE = RATE -1
        WHERE LOANID = R.LOANID;
    END LOOP;
    DMBS_OUTPUT.PUT_LINE('UPDATED');
END;

```

Scenario 2:

```

DECLARE
BEGIN
    FOR R IN (
        SELECT CUSTOMERID
        FROM CUSTOMERS
        WHERE BALANCE > 10000
    )
    LOOP
        UPDATE CUSTOMERS
        SET IsVIP = 'TRUE'
        WHERE CUSTOMERID=R.CUSTOMERID
    END LOOP;
    DMBS_OUTPUT.PUT_LINE('UPDATED');
END;

```

Scenario 3:

```
DECLARE
BEGIN
    FOR R IN (
        SELECT C.NAME, L.LOANID , L.DUEDATE
        FROM CUSTOMERS C JOIN LOANS L
        ON C.CUSTOMERID = L.CUSTOMERID
        WHERE L.DUEDATE BETWEEN SYSDATE AND SYSDATE + 30
    )
LOOP
    DBMS_OUTPUT.PUT_LINE
    (
        'REMINDER:' ||R.NAME||', YOUR LOAN'||R.LOANID||'IS DUE ON'||R.DUEDATE
    );
END LOOP;
END;
```

Exercise 3: Stored Procedures

```
1.CREATE OR REPLACE PROCEDURE MonthlyInterest
AS
BEGIN
    UPDATE Accounts
    SET Balance = Balance + (Balance *0.01)
    WHERE Accounttype='SAVINGS';
    DBMS_OUTPUT.PUT_LINE('MONTHLY INTEREST PROCESSED');
END;
```


2. CREATE OR REPLACE PROCEDURE EMPLOYEEBONUS(D IN VARCHAR2,PERCENT IN NUMBER)

AS

BEGIN

 UPDATE EMPLOYEES

 SET SALARY = SALARY +(SALARY * PERCENT/100)

 WHERE DEPARTMENT = D;

 DBMS_OUTPUT.PUT_LINE('BONUS UPDATED');

END;

3. CREATE OR REPLACE PROCEDURE TRANSFER

(SOURCE IN NUMBER, TARGET IN NUMBER, AMOUNT IN NUMBER)

AS

 B NUMBER;

BEGIN

 SELECT BALANCE

 INTO B

 FROM ACCOUNTS

 WHERE ACCOUNTID=SOURCE;

 IF B>=AMOUNT THEN

 UPDATE ACCOUNTS

 SET BALANCE = BALANCE - AMOUNT

 WHERE ACCOUNTID = TARGET;

 DBMS_OUTPUT.PUT_LINE('TRANSFER DONE');

 ELSE

 DBMS_OUTPUT.PUT_LINE('INSUFFICIENT BALANCE');

 END IF;

EXCEPTION

```
WHEN NO_DATA_FOUND THEN

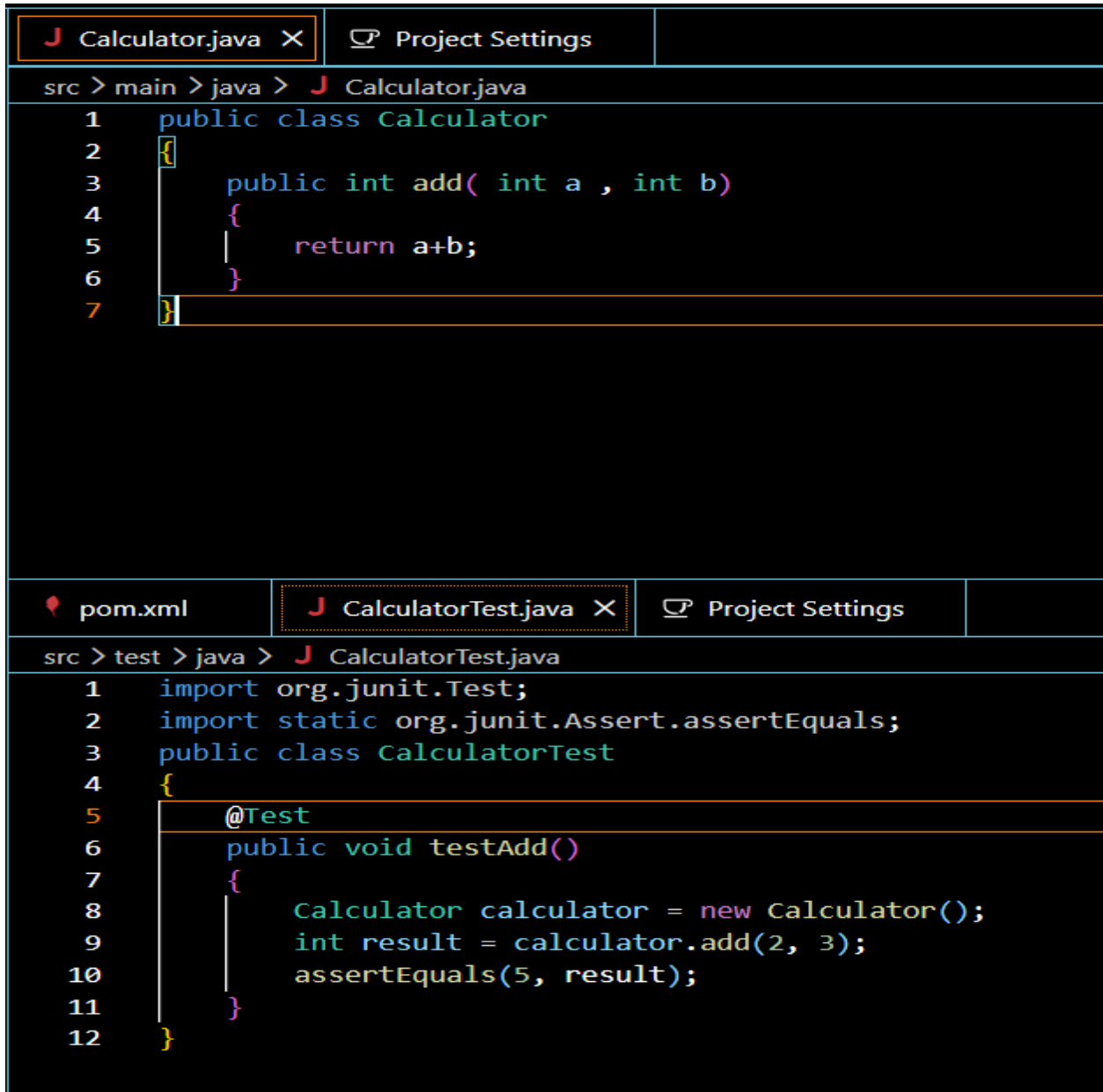
    DBMS_OUTPUT.PUT_LINE('ACCOUNT NOT FOUND');

WHEN OTHERS THEN

    DBMS_OUTPUT.PUT_LINE('ERROR: ' || SQLERRM);

END;
```

Exercise 1: Setting Up JUnit



The screenshot shows an IDE with two tabs: **Calculator.java** and **CalculatorTest.java**. The **Calculator.java** tab is active, showing the following code:

```
src > main > java > Calculator.java
1 public class Calculator
2 {
3     public int add( int a , int b)
4     {
5         return a+b;
6     }
7 }
```

The **CalculatorTest.java** tab is also visible, showing the following code:

```
src > test > java > CalculatorTest.java
1 import org.junit.Test;
2 import static org.junit.Assert.assertEquals;
3 public class CalculatorTest
4 {
5     @Test
6     public void testAdd()
7     {
8         Calculator calculator = new Calculator();
9         int result = calculator.add(2, 3);
10        assertEquals(5, result);
11    }
12 }
```

```

3-3.5.4.jar (12 kB at 210 kB/s)
[INFO] -----
[INFO] T E S T S
[INFO] -----
[INFO] Running CalculatorTest
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.111 s -- in CalculatorTest
[INFO] Results:
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 14.440 s
[INFO] Finished at: 2026-06-23T19:45:21+05:30
[INFO] -----
PS C:\Users\USER\Downloads\cognizant\week1\JUnitExercise1>

```

Exercise 3: Assertions in JUnit

```

.java | J AssertionsTest.java X | J CalculatorTest.java
src > test > java > J AssertionsTest.java
1  import org.junit.Test;
2  import static org.junit.Assert.*;
3  public class AssertionsTest
4  {
5      @Test
6      public void testAssertions()
7      {
8          assertEquals(5, 2 + 3);
9          assertTrue(5 > 3);
10         assertFalse(5 < 3);
11         assertNull(null);
12         assertNotNull(new Object());
13     }
14 }

```

```

FO] --- surefire-3.5.4:test (default-test) @ JUnitExercise1 ---
FO] Using auto detected provider org.apache.maven.surefire.junit4.JUnit4Provider
FO] -----
FO] T E S T S
FO] -----
FO] Running AssertionsTest
FO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.109 s -- in AssertionsTest
FO] Running CalculatorTest
FO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0 s -- in CalculatorTest
FO] Results:
FO] Tests run: 2, Failures: 0, Errors: 0, Skipped: 0
FO] -----
FO] BUILD SUCCESS
FO] -----
FO] Total time: 3.877 s
FO] Finished at: 2026-06-23T20:03:24+05:30
FO] -----
C:\Users\USER\Downloads\cognizant\week1\JUnitExercise1>

```

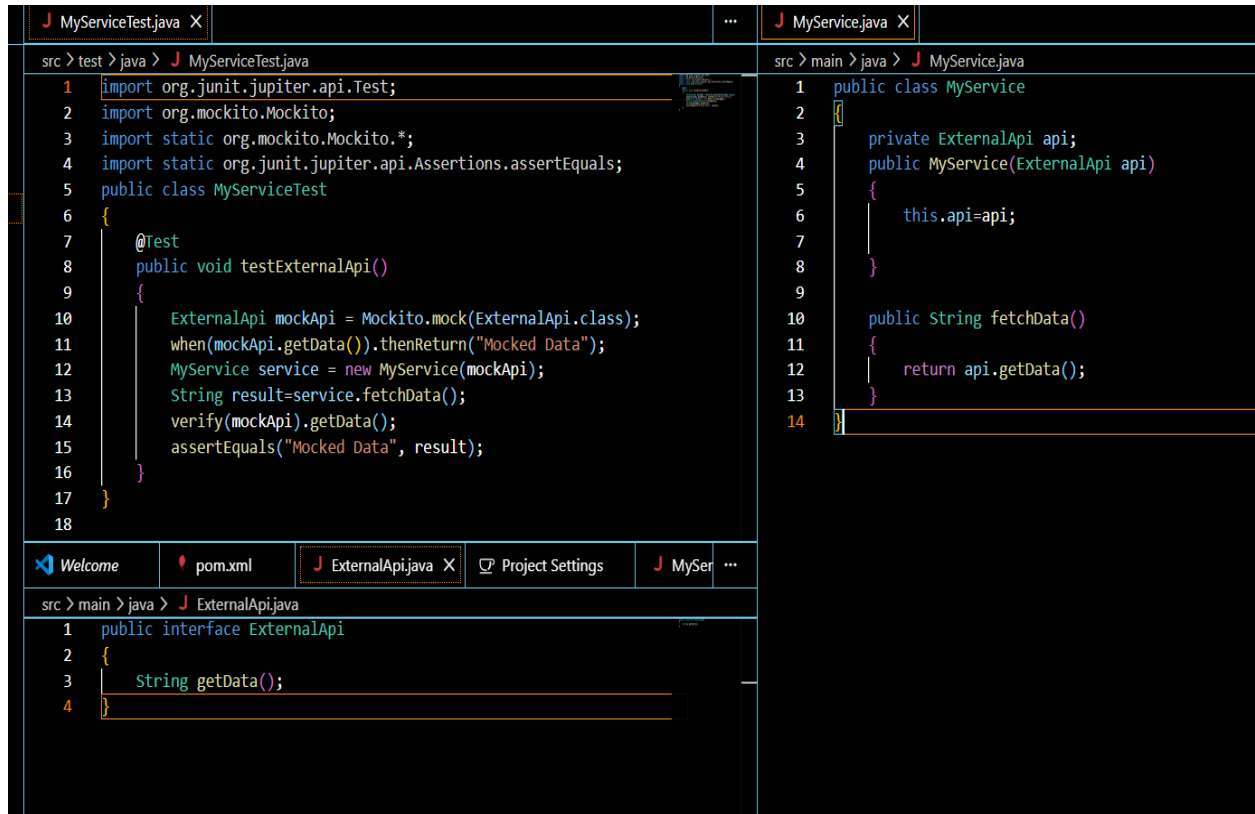
Exercise 4: Arrange-Act-Assert (AAA) Pattern, Test Fixtures, Setup and Teardown Methods in JUnit

```
J AssertionsTest.java | J CalculatorTest.java | Project Settings | ... | J Calculator.java
src > test > java > J CalculatorTest.java
1 import org.junit.Before;
2 import org.junit.After;
3 import org.junit.Test;
4 import static org.junit.Assert.*;
5 public class CalculatorTest
6 {
7     private Calculator calculator;
8
9     @Before
10    public void setUp()
11    {
12        calculator = new Calculator();
13        System.out.println("Setup executed");
14    }
15
16    @After
17    public void tearDown()
18    {
19        calculator = null;
20        System.out.println("Teardown executed");
21    }
22
23    @Test
24    public void testAdd()
25    {
26        int a = 8;
27        int b = 2;
28        int result = calculator.add(a, b);
29        assertEquals(10, result);
30    }
31
32    @Test
33    public void testSubtract()
```

```
src > main > java > J Calculator.java
1 public class Calculator
2 {
3     public int add( int a , int b)
4     {
5         return a+b;
6     }
7     public int subtract( int a , int b)
8     {
9         return a-b;
10    }
11 }
```

```
INFO] Compiling 2 source files with javac [debug target 1.8] to target\test-classes
[WARNING] bootstrap class path not set in conjunction with -source 8
INFO] --- surefire:3.5.4:test (default-test) @ JUnitExercise1 ---
INFO] Using auto detected provider org.apache.maven.surefire.junit4.JUnit4Provider
INFO]
INFO] -----
INFO] T E S T S
INFO] -----
INFO] Running AssertionsTest
INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.106 s -- in AssertionsTest
INFO] Running CalculatorTest
INFO] Setup executed
INFO] Teardown executed
INFO] Setup executed
INFO] Teardown executed
INFO] Tests run: 2, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.004 s -- in CalculatorTest
INFO]
INFO] Results:
INFO]
INFO] Tests run: 3, Failures: 0, Errors: 0, Skipped: 0
INFO]
INFO] -----
INFO] BUILD SUCCESS
INFO] -----
INFO] Total time: 3.901 s
INFO] Finished at: 2026-06-23T20:08:24+05:30
```

Exercise 1: Mocking and Stubbing



The screenshot shows an IDE with three open Java files. The top-left pane shows `MyServiceTest.java` with a test method `testExternalApi()` that uses Mockito to mock `ExternalApi`. The top-right pane shows `MyService.java` with a constructor that takes an `ExternalApi` and a `fetchData()` method that returns `api.getData()`. The bottom pane shows `ExternalApi.java` with a single method `getData()`.

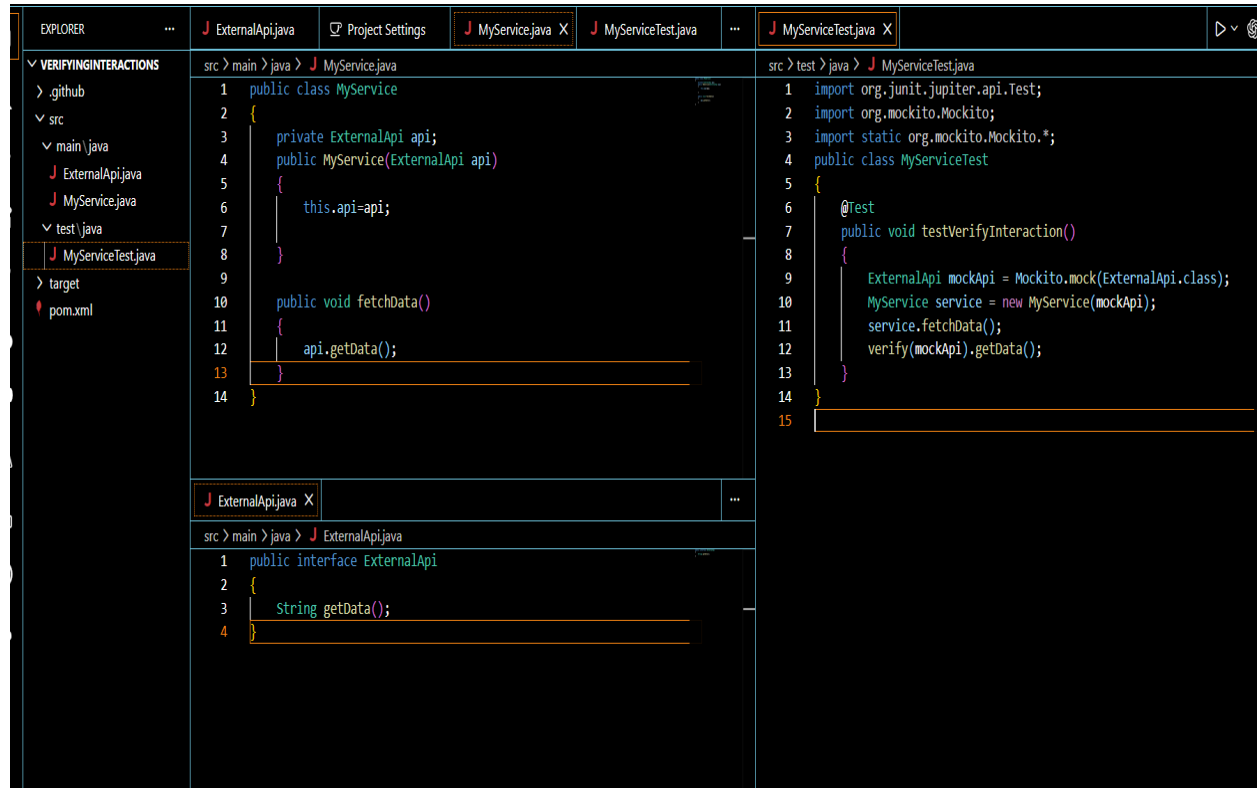
```
src > test > java > J MyServiceTest.java
1 import org.junit.jupiter.api.Test;
2 import org.mockito.Mockito;
3 import static org.mockito.Mockito.*;
4 import static org.junit.jupiter.api.Assertions.assertEquals;
5 public class MyServiceTest
6 {
7     @Test
8     public void testExternalApi()
9     {
10         ExternalApi mockApi = Mockito.mock(ExternalApi.class);
11         when(mockApi.getData()).thenReturn("Mocked Data");
12         MyService service = new MyService(mockApi);
13         String result=service.fetchData();
14         verify(mockApi).getData();
15         assertEquals("Mocked Data", result);
16     }
17 }
18

src > main > java > J MyService.java
1 public class MyService
2 {
3     private ExternalApi api;
4     public MyService(ExternalApi api)
5     {
6         this.api=api;
7     }
8
9     public String fetchData()
10    {
11        return api.getData();
12    }
13 }
14

src > main > java > J ExternalApi.java
1 public interface ExternalApi
2 {
3     String getData();
4 }
```

```
atform-launcher-1.10.2.jar
ownloaded from central: https://repo.maven.apache.org/maven2/org/junit/platform/junit-pla
tform-launcher-1.10.2.jar (184 kB at 2.9 MB/s)
INFO]
INFO] -----
INFO]  T E S T S
INFO] -----
INFO] Running MyServiceTest
penJDK 64-Bit Server VM warning: Sharing is only supported for boot loader classes because
appended
INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.991 s -- in MyServ
INFO]
INFO] Results:
INFO]
INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0
INFO]
INFO] -----
INFO] BUILD SUCCESS
INFO] -----
INFO] Total time: 5.275 s
INFO] Finished at: 2026-06-23T20:37:40+05:30
INFO] -----
S C:\Users\USER\Downloads\cognizant\week1\VerifyingInteractions> |
```

Exercise 2: Verifying Interactions



```
Downloaded from central: https://repo.maven.apache.org/maven2/org/junit/platform/junit-platform-launcher-1.10.2.jar (184 kB at 163 kB/s)
[INFO] -----
[INFO] T E S T S
[INFO] -----
[INFO] Running MyServiceTest
OpenJDK 64-Bit Server VM warning: Sharing is only supported for boot loader classes
appended
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 1.278 s -- in
[INFO] Results:
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 7.141 s
[INFO] Finished at: 2026-06-23T20:28:07+05:30
[INFO] -----
PS C:\Users\USER\Downloads\cognizant\week1\VerifyingInteractions> |
```

Exercise 1: Logging Error Messages and Warning Levels

```
J LoggingExample.java X
src > main > java > J LoggingExample.java
1  import org.slf4j.Logger;
2  import org.slf4j.LoggerFactory;
3  public class LoggingExample
4  {
5      private static final Logger logger= LoggerFactory.getLogger(LoggingExample.class);
6      public static void main(String[] args)
7      {
8          logger.info("This is an info message");
9          logger.warn("This is a warning message");
10         logger.error("This is an error message");
11     }
12 }
```

```
(267 kB at 2.2 MB/s)
nloading from central: https://repo.maven.apache.org/maven2/org/codehaus/plexus/plexus-compo
s-component-annotations-2.1.1.jar
nloading from central: https://repo.maven.apache.org/maven2/org/apache/commons/commons-exec/
nloaded from central: https://repo.maven.apache.org/maven2/org/codehaus/plexus/plexus-compon
-component-annotations-2.1.1.jar (4.1 kB at 56 kB/s)
nloaded from central: https://repo.maven.apache.org/maven2/org/apache/commons/commons-exec/1
kB at 285 kB/s)
47:37.828 [LoggingExample.main()] INFO LoggingExample - This is an info message
47:37.831 [LoggingExample.main()] WARN LoggingExample - This is a warning message
47:37.831 [LoggingExample.main()] ERROR LoggingExample - This is an error message
FO] -----
FO] BUILD SUCCESS
FO] -----
FO] Total time: 1.495 s
FO] Finished at: 2026-06-23T20:47:37+05:30
FO] -----
C:\Users\USER\Downloads\cognizant\week1\LoggingExercise> |
```